

Segurança em Computação

INE5429-07208

Thaís Bardini Idalino

Criptografia assimétrica

Sumário

- **Contexto histórico e definições básicas**
- Algoritmo de troca de chaves DH
- Criptossistema RSA
- Criptografia em curvas elípticas (ECC)
- Considerações finais
- Atividades práticas

Criptografia Simétrica

- **Eficiente:** a base de diversos algoritmos vistos consiste em operações simples, como ou-exclusivo, permutações e substituições
- **Segura:** algoritmos como o AES são resistentes a técnicas de criptoanálise conhecidas
- **Chaves pequenas:** qualquer coisa pode ser chave
- **Desafio:** como compartilhar a chave secreta para que emissor e receptor consigam cifrar e decifrar?

Criptografia de chave pública (assimétrica)

- Uma das maiores evoluções na história da criptografia
- Baseada em funções matemáticas ao invés de substituições e permutações
- São **assimétricas**: utilizam duas chaves diferentes, o que tem consequências profundas nas áreas de confidencialidade, distribuição de chaves, autenticação, etc.
- **Desafio**: as técnicas existentes ainda são menos eficientes do que a criptografia simétrica.
- **Combinação**: utilização de uma técnica assimétrica para o cálculo de chaves simétricas

Características

Um criptossistema de chave pública possui os seguintes ingredientes:

1. texto claro
2. algoritmo de cifragem
3. par de chaves (uma pública (pk) e outra privada (sk))
4. texto cifrado
5. algoritmo de decifragem



Características

- Cada usuário gera um par de chaves
 - Uma pública, disponibilizada em algum local público
 - Uma privada, que nunca deve ser compartilhada
- Se Bob quer enviar uma mensagem **confidencial** para Alice:
 - ele cifra a mensagem com a **chave pública** da Alice,
 - ao receber a mensagem, Alice decifra com sua **chave privada**.
 - Ela é a única que tem acesso à chave e pode decifrar a mensagem.

Requisitos

- Computacionalmente fácil gerar um par de chaves
- Computacionalmente fácil para o remetente operar com a chave pública (pk)
 - $c = E(m, pk)$
- Fácil para o destinatário operar com a chave privada (sk)
 - $m = D(c, sk) = D(E(m, pk), sk)$
- Computacionalmente inviável que um adversário determine sk a partir de pk
- Computacionalmente inviável que um adversário recupere a m conhecendo pk e c
- Impossível forjar uma assinatura conhecendo a mensagem e pk
 - veremos assinaturas digitais nas próximas aulas

Aplicações

- Podemos classificar criptossistemas de chaves públicas em três categorias:
 - Cifragem/decifragem
 - Assinatura digital
 - Troca de chaves
- Alguns algoritmos servem para os três propósitos, outros apenas para um ou dois deles.

Table 9.3 Applications for Public-Key Cryptosystems

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

Imagem: W. Stallings. *Cryptography and network security*. Cap 9.3

Sumário

- Contexto histórico e definições básicas
- **Algoritmo de troca de chaves DH**
- Criptossistema RSA
- Criptografia em curvas elípticas (ECC)
- Considerações finais
- Atividades práticas

Diffie-Hellman (1976)

- Criptografia de chave pública foi idealizada por Diffie e Hellman (1976) em Stanford.
- Publicaram o primeiro e mais simples algoritmo de chave pública.

Troca de chaves de Diffie–Hellman:

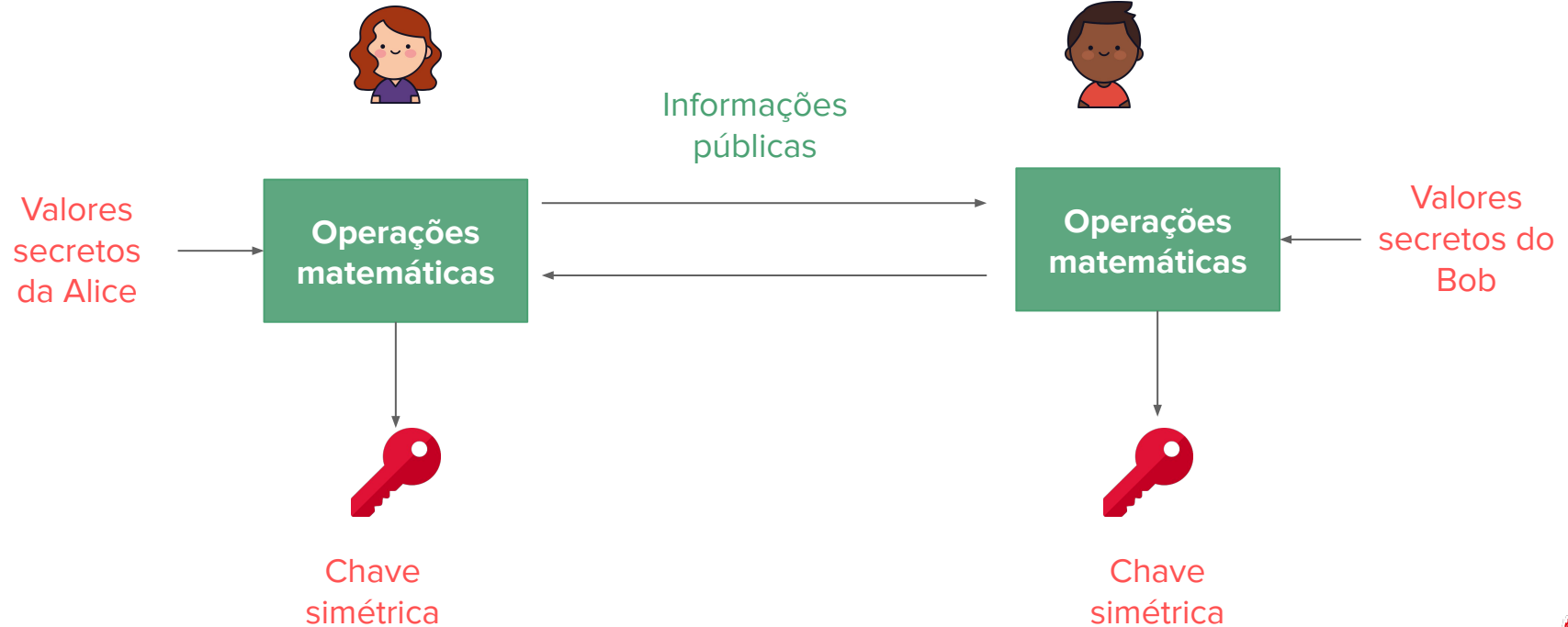
- **Objetivo:** permitir que dois usuários consigam trocar uma chave de maneira segura para depois utilizá-la na cifragem simétrica de mensagens.
- **Segurança:** O algoritmo depende da dificuldade de calcular logaritmos discretos



Imagem: <https://tinyurl.com/diffie-hellman>

“what good would it do after all to develop impenetrable cryptosystems, if their users were forced to share their keys with a KDC that could be compromised by either burglary or subpoena?” - Diffie

Acordo de chaves



Relembrando

- Seja p um número primo, considere o grupo multiplicativo $\mathbb{Z}_p^* = \{1, 2, 3, \dots, p-1\}$
- Uma raiz primitiva a de p é um número tal que a sequência:

$$a \bmod p, a^2 \bmod p, \dots, a^{p-1} \bmod p$$

gera todos os números de $\mathbb{Z}_p^*$ em alguma ordem

- Para qualquer inteiro b e raiz primitiva a de um número primo p , podemos encontrar um único expoente x tal que $b = a^x \bmod p$.
 - x é o **logaritmo discreto** de b
 - geralmente, é muito difícil calcular/encontrar x

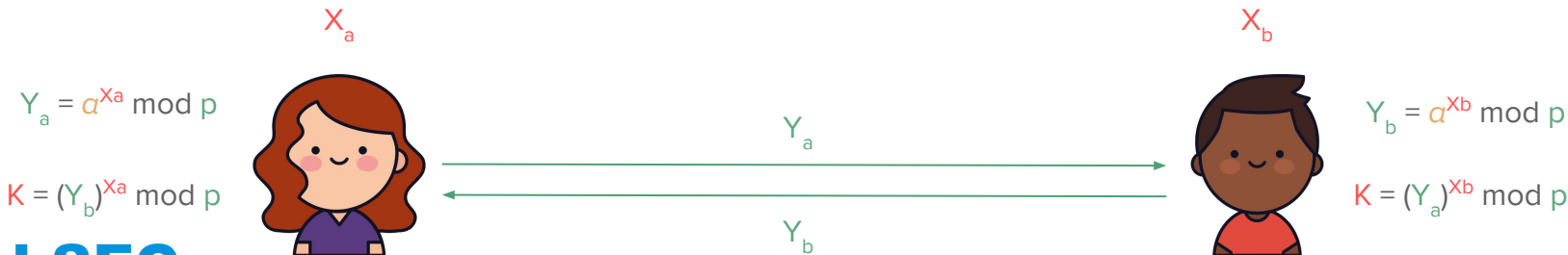
A troca de chaves Diffie-Hellman

Parâmetros

- **Parâmetros públicos:**
 - p número primo
 - a raiz primitiva de $\mathbb{Z}_p^*$
- **Parâmetros privados:**
 - X_a e X_b números aleatórios $< p$

Algoritmo:

- **Cálculo dos valores públicos:**
 - Alice calcula: $Y_a = a^{X_a} \bmod p$
 - Bob calcula: $Y_b = a^{X_b} \bmod p$
- **Cálculo do segredo:**
 - Alice calcula: $K = (Y_b)^{X_a} \bmod p$
 - Bob calcula: $K = (Y_a)^{X_b} \bmod p$



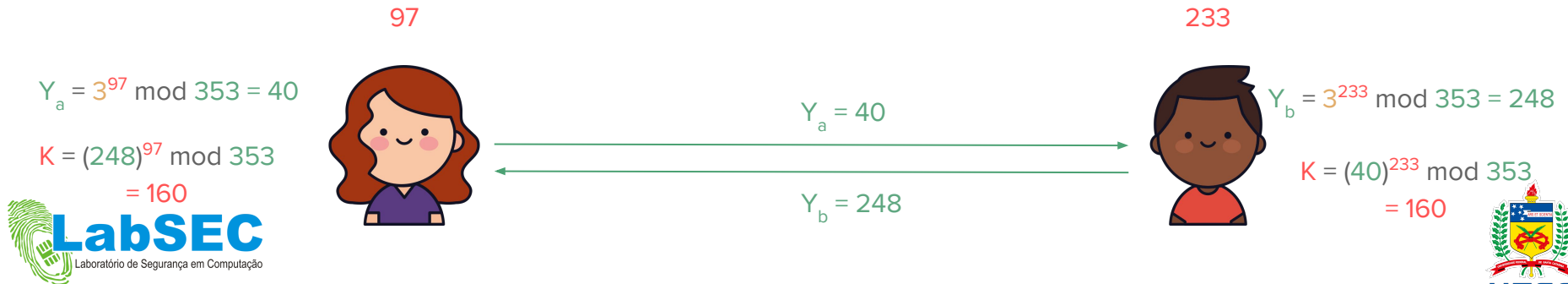
Exemplo

Parâmetros:

- $p = 353$, $a = 3$
- Alice calcula $X_a = 97$
- Bob calcula $X_b = 233$

Explicação:

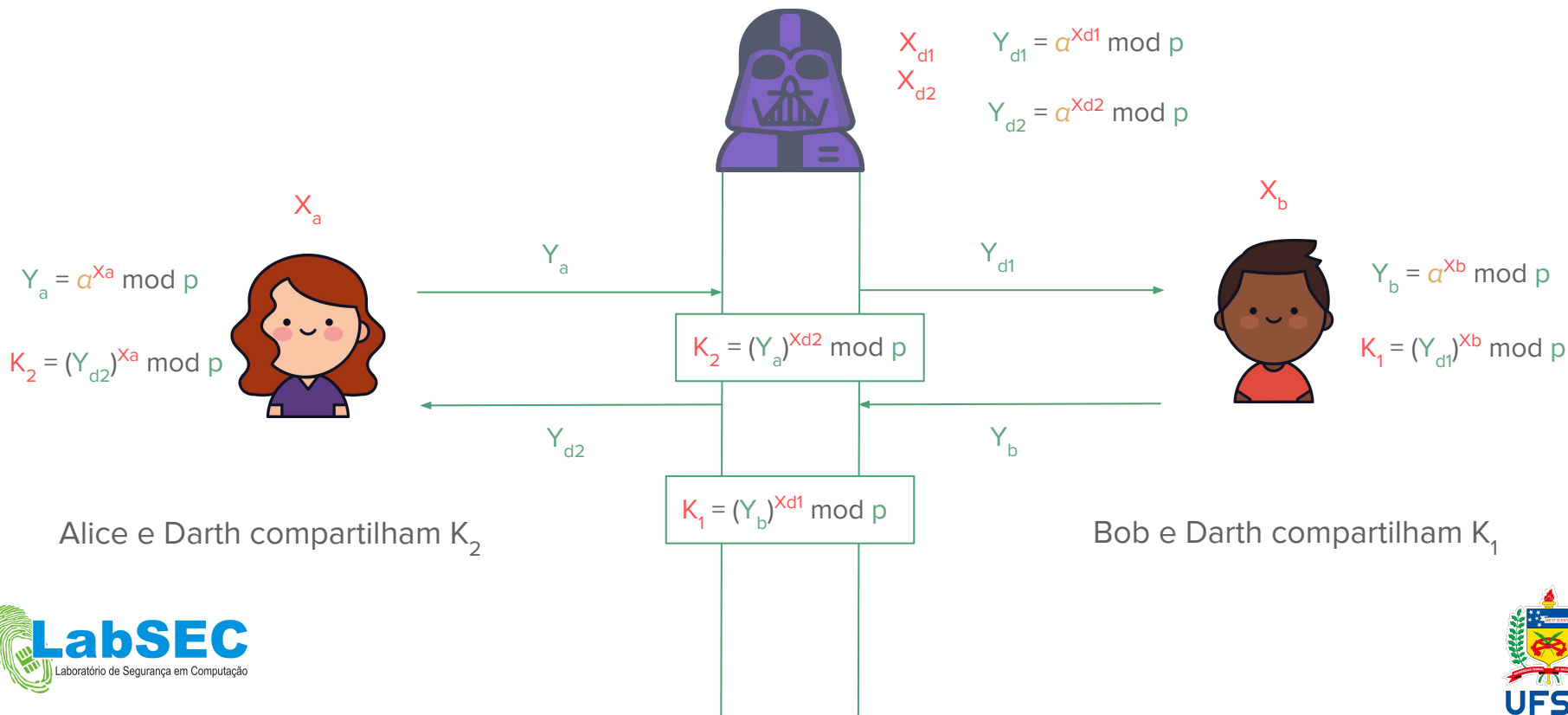
- Alice calcula: $Y_a = a^{X_a} \bmod p$
- Bob calcula: $Y_b = a^{X_b} \bmod p$
- Alice calcula: $(Y_b)^{X_a} \bmod p = (a^{X_b})^{X_a} \bmod p$
- Bob calcula: $(Y_a)^{X_b} \bmod p = (a^{X_a})^{X_b} \bmod p$



Segurança do algoritmo

- Apenas Alice e Bob conhecem os valores X_a e X_b , respectivamente
- é relativamente simples calcular exponenciações modulares
- é computacionalmente inviável resolver o **problema do logaritmo discreto** para valores grandes o suficiente
 - Ou seja, para $Y_a = a^{X_a} \bmod p$ e $Y_b = a^{X_b} \bmod p$
 - é inviável descobrir X_a ou X_b a partir de Y_a ou Y_b
- Ou seja, apenas Alice e Bob conseguem calcular K
 - portanto, esse valor pode ser utilizado entre eles como uma chave simétrica.
- O que significa "grande o suficiente"?
 - Atualmente, o primo p precisa ter, pelo menos 2048 bits

O ataque Man-in-the-Middle



O ataque Man-in-the-Middle

- O protocolo só é vulnerável a esse ataque pois não há autenticação dos participantes
- Essa vulnerabilidade pode ser resolvida utilizando assinaturas digitais e certificados digitais
 - a ser explorado nas próximas aulas

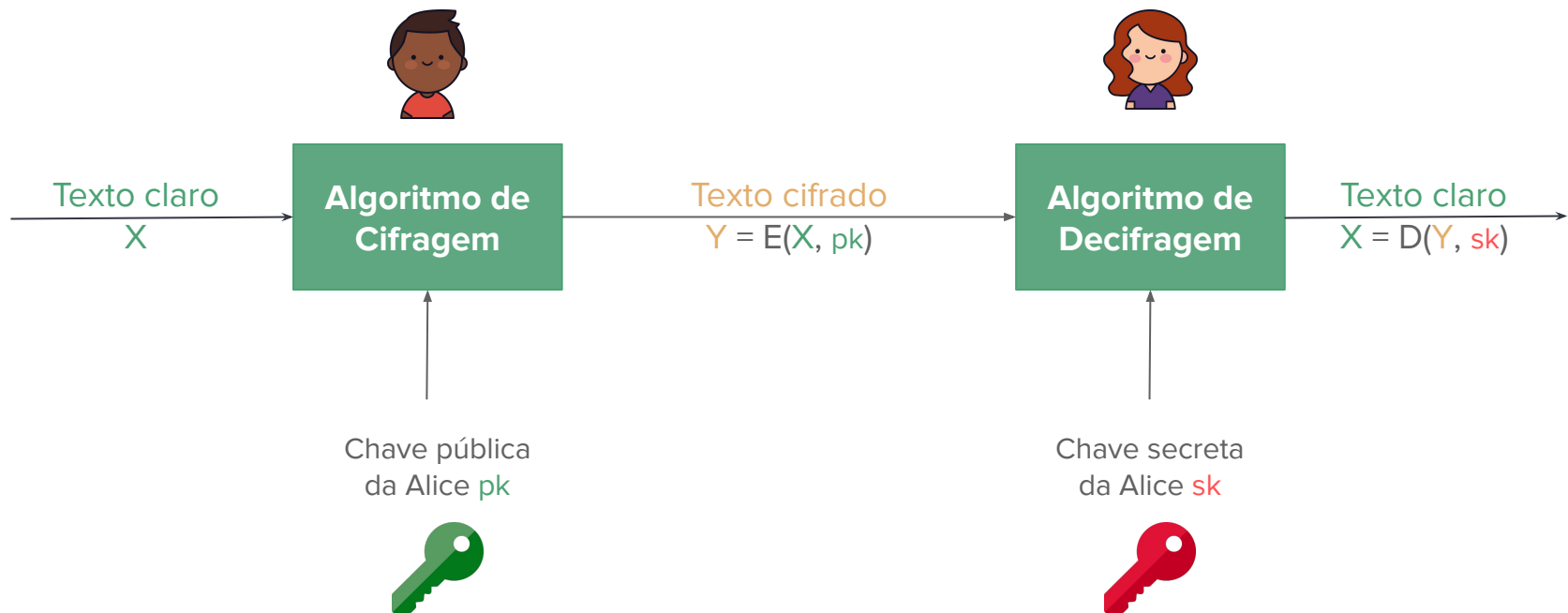
DH na prática

- TLS/SSL: protocolo utilizado na proteção de tráfego web (HTTPS), frequentemente utiliza DH para estabelecer conexões seguras. Importante na proteção de dados sensíveis trafegados pela internet
- SSH: provê acesso remoto seguro à sistemas computacionais. Utiliza DH para criar uma chave entre cliente e servidor para a transmissão de dados cifrados.
- VPNs: utilizadas para estabelecer um canal de comunicação seguro na internet. Utiliza DH para estabelecer um túnel seguro entre cliente e o servidor VPN
- Logjam (2015): permitiu ataques man-in-the-middle que forçam o servidor a escolher parâmetros inseguros de 512 bits. Com isso, atacantes conseguem ler e modificar qualquer dado sendo transmitido naquela conexão.

Sumário

- Contexto histórico e definições básicas
- Algoritmo de troca de chaves DH
- **Criptossistema RSA**
- Criptografia em curvas elípticas (ECC)
- Considerações finais
- Atividades práticas

Voltando para cifragem/decifragem



O criptossistema RSA

- **1976:** Diffie e Hellman desafiaram a comunidade científica a criar um algoritmo que atendesse aos requisitos da criptografia de chave pública (assimétrica)
- **1977-1978:** Um dos primeiros criptossistemas foi desenvolvido por Rivest, Shamir, e Adleman, o RSA.
- **Hoje:** o RSA é a abordagem de propósito geral mais amplamente aceita e implementada para criptografia de chave pública



Ronald L. Rivest



Adi Shamir



Leonard M. Adleman

Imagem: <http://www.ams.org/notices/200307/comm-turing.pdf>

RSA - Overview

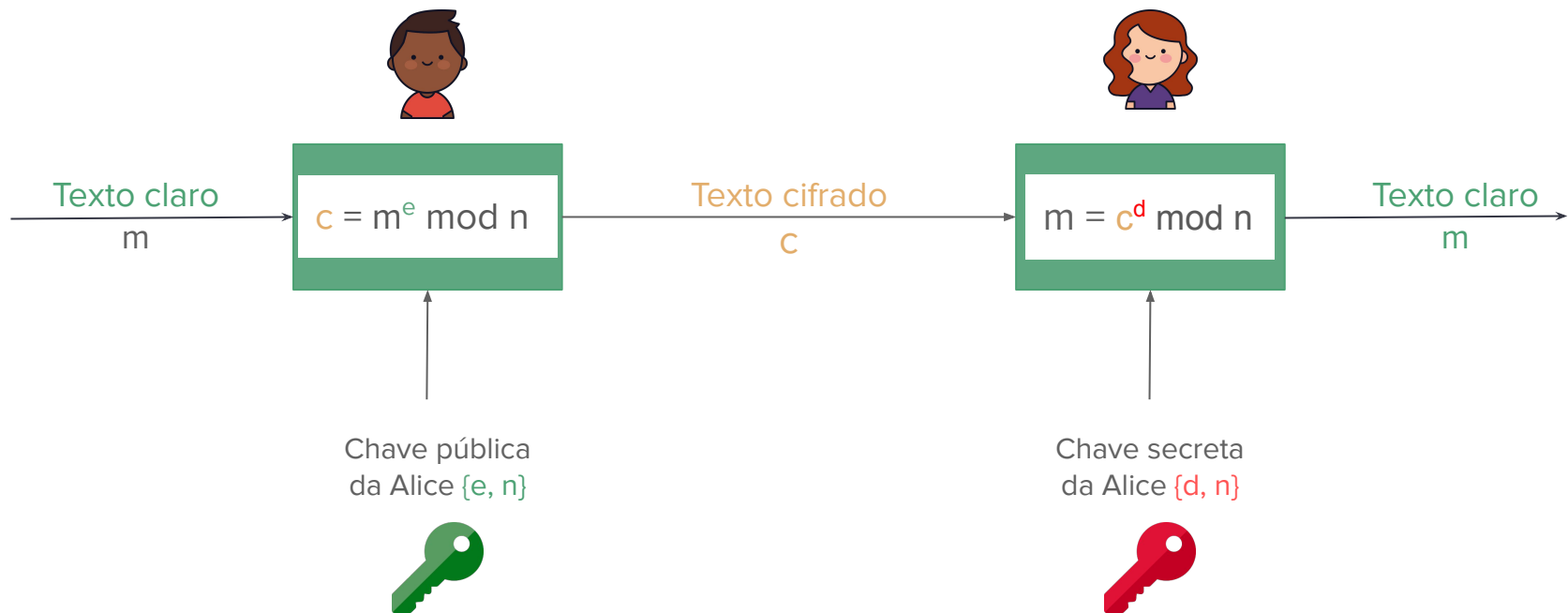
- Necessidade de teoria de números
 - Utiliza números primos p e q
 - Cálculos em $\mathbb{Z}_n$, onde $n = pq$
 - uma chave é a inversa multiplicativa da outra
 - **chave pública** = $\{e, n\}$, **chave privada** = $\{d, n\}$
 - segurança baseada na dificuldade de fatorar n
- Cifragem e decifragem
 - $c = E(m, e) = m^e \bmod n$ (ou seja, $c \in \mathbb{Z}_n$)
 - $m = D(c, d) = c^d \bmod n$
- Tamanho das chaves
 - o valor n tem entre 1024 e 2048 bits
 - cada primo tem entre 512 e 1024 bits

Geração de chaves

- p, q primos (valores **secretos** e escolhidos)
- $n = p \cdot q$ (valor **público** e calculado)
- $\phi(n) = \phi(pq) = \phi(p)\phi(q) = (p-1)(q-1)$ (valor **secreto**, só quem sabe p e q consegue calcular)
- As chaves e e d são inversas multiplicativas módulo $\phi(n)$
 - Escolhemos o e tal que $\text{mdc}(\phi(n), e) = 1$ e $1 < e < \phi(n)$ (garantimos que e terá inversa)
 - Calculamos d tal que $ed \equiv 1 \pmod{\phi(n)}$ (ou seja, $ed \bmod \phi(n) = 1$ ou ainda $ed = k\phi(n) + 1$)
 - Ou seja, d é a inversa multiplicativa de e módulo $\phi(n)$ (encontramos com Algoritmo de Euclides Estendido)



Cifragem e Decifragem



Exemplo

Geração de Chaves

- $p = 17$ e $q = 11$
- $n = p \times q = 17 \times 11 = 187$
- $\phi(n) = (p - 1)(q - 1) = 16 \times 10 = 160$
- Escolhemos $e = 7$
 - $\text{mdc}(160, 7) = 1$
 - $1 < 7 < 160$
- Calculamos $d = 23$ (Euclides estendido)
 - $ed = 161 \equiv 1 \pmod{160}$
- Chave Pública = $\{7, 187\}$
- Chave Privada = $\{23, 187\}$

Cifragem

- Texto Claro = 88
- $c = m^e = 88^7 \pmod{187} = 11$

Decifragem

- $c^d \pmod{187} = 11^{23} \pmod{187} = 88$

Requisitos

1. É possível encontrar valores para e , d , n tal que $m^{ed} \bmod n = m$ para todo $m < n$.
2. É relativamente fácil calcular $m^e \bmod n$ e $c^d \bmod n$ para para todo $m < n$.
3. É inviável determinar d dados e e n .

Corretude (requisito 1)

- Utilizando o teorema de Euler e inversas multiplicativas.
- **Lembrete:**
 - **Cifragem:** $c = m^e \bmod n$
 - **Decifragem:** $m = c^d \bmod n$
 - note que $c^d = (m^e)^d$
- **Detalhes matemáticos:**
 - como $e \cdot d \bmod \phi(n) = 1$, podemos escrever como $e \cdot d = k\phi(n) + 1$ (inversa multiplicativa módulo $\phi(n)$)
 - então, $c^d = (m^e)^d = m^{k\phi(n) + 1} = (m^{\phi(n)})^k \times m$
 - Pelo **teorema de Euler**, temos $m^{\phi(n)} \equiv 1 \pmod{n}$
 - então, $c^d \equiv (m^{\phi(n)})^k \times m \equiv m \pmod{n}$
 - portanto, a decifragem $m^{ed} \bmod n$ é capaz de corretamente obter a mensagem original para qualquer $m < n$.

Aspectos computacionais (requisito 2)

- Cifragem e Decifragem utilizam exponenciação modular
- **Opção 1:** exponenciação feita nos inteiros e depois aplicar o módulo
 - valores intermediários **muito** grandes
 - **Exemplo:** $x^{11} = x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x$
- **Opção 2:** utilizar a propriedade $[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$
 - valores intermediários reduzidos
 - calcular a exponenciação utilizando *repeated squares*
 - $x^{11} = x \times x^2 \times x^8$ (note que $x^2 = x \times x$, $x^4 = x^2 \times x^2$ e $x^8 = x^4 \times x^4$)
 - calculamos $x \bmod n$, $x^2 \bmod n$, $x^4 \bmod n$, $x^8 \bmod n$
 - depois calculamos $[(x \bmod n) \times (x^2 \bmod n) \times (x^8 \bmod n)] \bmod n$

Aspectos computacionais (requisito 2)

- Escolhas de e para eficiência:
 - $65537 = 2^{16} + 1$ ou 17 ou 3
 - Valores com apenas dois bits “1”
 - garantem aceleração na multiplicação
- d precisa ser grande para evitar força bruta
- Gerar chaves pode ser demorado
 - precisamos do teste de primalidade várias vezes em um número muito grande
 - Ao fixar e , podemos precisar recalcular p, q algumas vezes

Segurança (requisito 3)

- **Força bruta:** tentar todas as possibilidades de chave privada
- **Ataques matemáticos:** diversas técnicas, com objetivo de, por exemplo, fatorar n de maneira mais eficiente
- **Timing attacks:** explora variações de tempo na execução de uma operação criptográfica
- **Ataques e hardware:** induzir erros de hardware em um módulo criptográfico para descobrir informações sobre a chave
- **Chosen ciphertext attack:** o atacante consegue acesso à mensagem original correspondente a uma cifra para descobrir informações sobre a chave

A chance de sucesso depende da implementação do algoritmo e da escolha dos parâmetros vulneráveis.

Atacando o RSA matematicamente

- Existem 3 formas:
 - Fatorar n e descobrir p e q
 - permite o cálculo de $\phi(n) = (p - 1)(q - 1)$ e a determinação de $d \equiv e^{-1} \pmod{\phi(n)}$
 - Determinar $\phi(n)$ diretamente, sem a necessidade de p e q .
 - assim como antes, permite a determinar d .
 - Determinar d diretamente.
- Criptanalistas tem focado na fatoração de n .
 - que é muito difícil quando p e q são grandes o suficiente.
- RSA Laboratories lançou desafios de fatoração com tamanhos variados de n .
- Técnicas envolvem algoritmos elaborados
 - Ex:** quadratic sieve, number field sieve.

Table 9.5 Progress in RSA Factorization

Number of Decimal Digits	Number of Bits	Date Achieved
100	332	April 1991
110	365	April 1992
120	398	June 1993
129	428	April 1994
130	431	April 1996
140	465	February 1999
155	512	August 1999
160	530	April 2003
174	576	December 2003
200	663	May 2005
193	640	November 2005
232	768	December 2009

Imagem: W. Stallings. *Cryptography and network security*. Cap 9.2

RSA e os computadores quânticos

- O algoritmo de Shor é um algoritmo quântico desenvolvido em 1994, capaz de fatorar números inteiros grandes de forma eficiente
- Com ele, também seria possível calcular o logaritmo discreto eficientemente
- Se um computador quântico suficientemente grande for construído, esses algoritmos podem "quebrar" tanto o DH quanto o RSA
- Em um *draft* sobre a transição para criptografia pós-quântica, o NIST recomenda o desuso do RSA de 2048 bits após 2030 e proíbe o uso após 2035.

Aplicações do RSA

- **SSL/TLS:** usa RSA na autenticação da identidade de sites para o estabelecimento de conexões seguras;
- **Assinaturas Digitais:** Utilizadas para validar a autenticidade e integridade de documentos eletrônicos;
- **Distribuição Segura de Chaves:** usado em criptografia híbrida, para proteger a transmissão de chaves simétricas;
- entre muitas outras!

Sumário

- Contexto histórico e definições básicas
- Algoritmo de troca de chaves DH
- Criptossistema RSA
- **Criptografia em curvas elípticas (ECC)**
- Considerações finais
- Atividades práticas

Criptografia em Curvas Elípticas

- Resposta ao tamanho de chaves RSA
 - que tem crescido nos últimos anos
- Mesma segurança com chaves menores
- Computações mais eficientes
- Baseada em funções matemáticas simples de calcular mas difíceis de reverter

Table 9.3 Applications for Public-Key Cryptosystems

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

Definições básicas

- **Curva elíptica:** conjunto de pontos que satisfaz a equação $y^2 = x^3 + ax + b$
 - Chamamos esse conjunto de pontos (x,y) de $E(a,b)$, para dados a e b .
 - Variáveis e coeficientes estão restritos a valores em um conjunto especial de elementos (ex: $\mathbb{Z}_p$)
 - Também definimos um ponto no infinito O
- **Exemplo:**
 - Curva $E(-1,0)$ que satisfaz $y^2 = x^3 - x$ em $\mathbb{R}$
 - Curva $E(-1, b)$ que satisfaz $y^2 = x^3 - x + b$ em $\mathbb{R}$, para $b = 0, 1/10, 2/10, 3/10, 4/10, 5/10$.

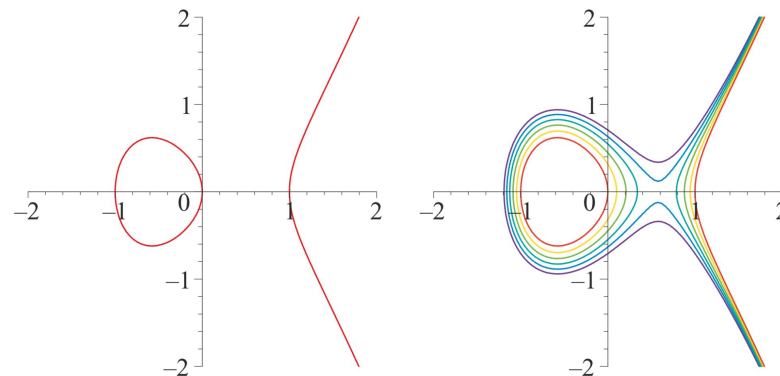


Imagem: $E(-1,0)$ e $E(-1, b)$.
Joachim von zur Gathen.
CryptoSchool. Cap 5

Definições básicas

- **Curva elíptica:** conjunto de pontos que satisfaz a equação $y^2 = x^3 + ax + b$
 - Chamamos esse conjunto de pontos (x,y) de $E(a,b)$, para dados a e b .
 - Variáveis e coeficientes estão restritos a valores em um conjunto especial de elementos (ex: $\mathbb{Z}_p$)
 - Também definimos um ponto no infinito O
- **Exemplo:**
 - Curva $E_{23}(1, 1)$ que satisfaz $y^2 = x^3 + x + 1$ em $\mathbb{Z}_{23}$.
 - $(0, 1) \in E_{23}(1, 1)$

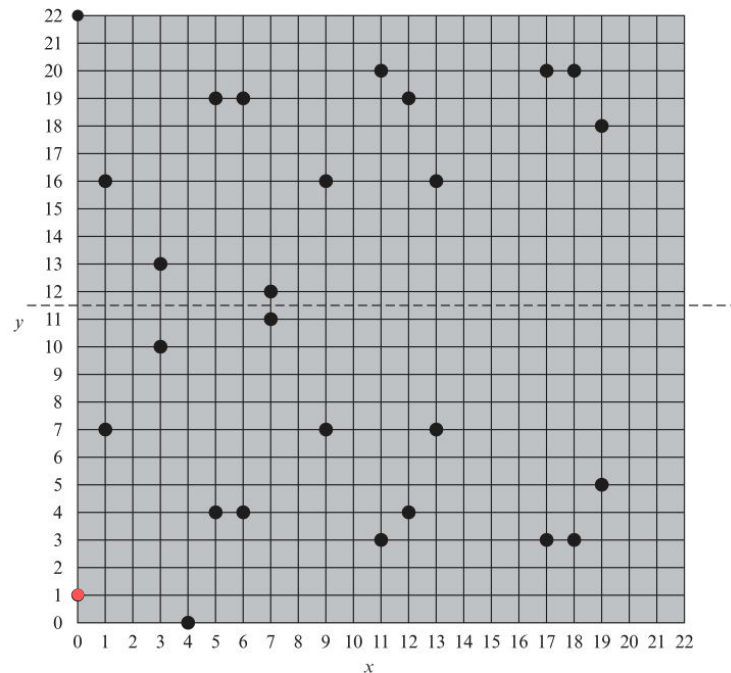


Figure 10.5 The Elliptic Curve $E_{23}(1, 1)$

Definições básicas

- **Curva elíptica:** conjunto de pontos que satisfaz a equação $y^2 = x^3 + ax + b$
 - Chamamos esse conjunto de pontos (x,y) de $E(a,b)$, para dados a e b .
 - Variáveis e coeficientes estão restritos a valores em um conjunto especial de elementos (ex: $\mathbb{Z}_p$)
 - Também definimos um ponto no infinito O
- **Exemplo:**
 - Curva $E_{23}(1, 1)$ que satisfaz $y^2 = x^3 + x + 1$ em $\mathbb{Z}_{23}$.

Table 10.1 Points (other than O) on the Elliptic Curve $E_{23}(1, 1)$

(0, 1)	(6, 4)	(12, 19)
(0, 22)	(6, 19)	(13, 7)
(1, 7)	(7, 11)	(13, 16)
(1, 16)	(7, 12)	(17, 3)
(3, 10)	(9, 7)	(17, 20)
(3, 13)	(9, 16)	(18, 3)
(4, 0)	(11, 3)	(18, 20)
(5, 4)	(11, 20)	(19, 5)
(5, 19)	(12, 4)	(19, 18)

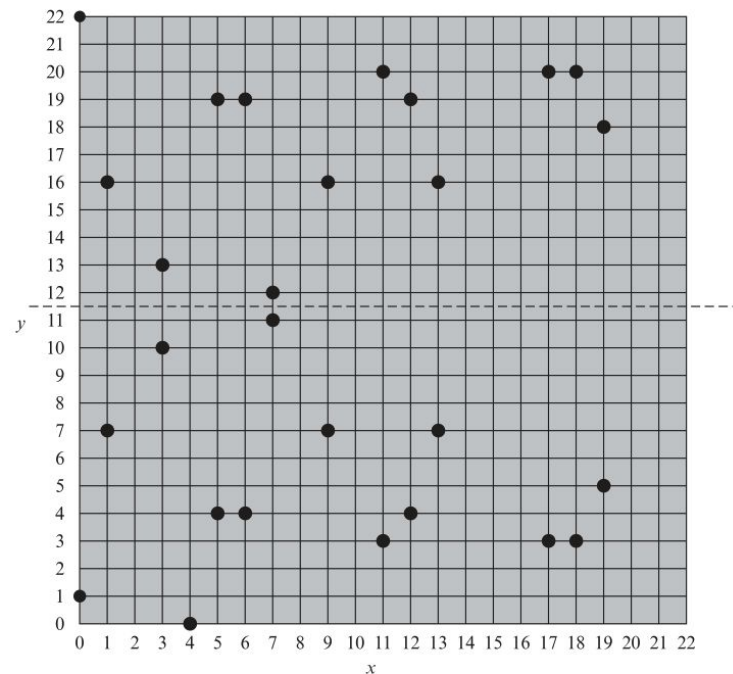


Figure 10.5 The Elliptic Curve $E_{23}(1, 1)$

Imagens: W. Stallings. *Cryptography and network security*. Cap 10.3

Propriedades

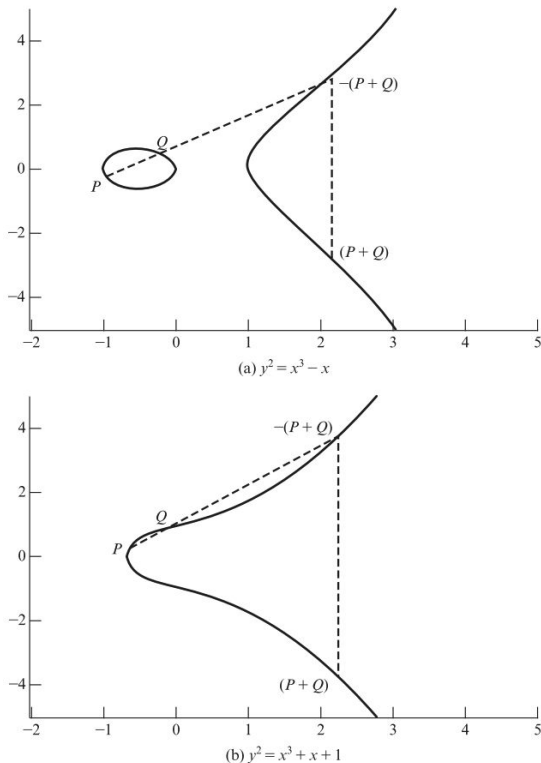


Figure 10.4 Example of Elliptic Curves

- $E(a,b)$: conjunto de pontos que satisfaz a equação $y^2 = x^3 + ax + b$
- **Soma** de dois pontos P e Q dão um outro ponto R na curva ($P+Q = R$).
- **Multiplicação** pode ser definida como somas consecutivas: $P+P = 2P$ e o resultado está na curva.
- $P + O = P$ (identidade)
- Se $P = (x,y)$, então $-P = (x, -y)$ (inversa)
- $P + (-P) = O$

Imagem: W. Stallings. *Cryptography and network security*. Cap 10.3

Criptografia em Curvas Elípticas

- A adição em ECC é a contrapartida da multiplicação modular no RSA.
- A multiplicação em ECC é a contrapartida da exponenciação modular.
- Precisamos de um **problema difícil** em ECC, assim como temos a fatoração ou o problema do logaritmo discreto.

Criptografia em Curvas Elípticas

- No protocolo Diffie-Hellman fazemos $Y = m^k \bmod p$
 - multiplicamos m por ele mesmo k vezes
 - um atacante precisa determinar k (log discreto)
- Em ECC, fazemos somas sucessivas
 - Calculamos $Q = kP$, onde Q e P são pontos em $E_p(a,b)$ (ou seja, em $\mathbb{Z}_p$) e $k < p$
 - é fácil calcular Q dado k e P
 - é difícil determinar k dado Q e P
 - Esse é o **problema do logaritmo discreto em curvas elípticas**
 - k é muito grande, tornando força-bruta inviável

Troca de chaves em ECC

Parâmetros

- **Parâmetros públicos:**
 - q número primo ou na forma 2^m
 - parâmetros a e b para a curva $E_q(a,b)$
 - ponto $G = (x_1, y_1)$ em $E_q(a,b)$ cuja ordem é um número grande n
 - ordem: menor inteiro n tal que $nG = O$
- **Parâmetros privados:**
 - n_a e n_b números aleatórios $< n$

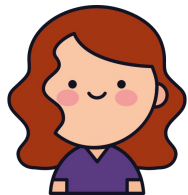
Algoritmo:

- **Cálculo dos valores públicos:**
 - Alice calcula: $P_a = n_a \times G$
 - Bob calcula: $P_b = n_b \times G$
- **Cálculo do segredo:**
 - Alice calcula: $K = n_a \times P_b$
 - Bob calcula: $K = n_b \times P_a$

$$P_a = n_a \times G$$

$$K = n_a \times P_b$$

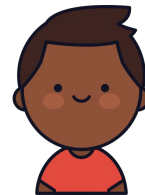
LabSEC
Laboratório de Segurança em Computação



n_a

P_a

P_b



n_b

$$P_b = n_b \times G$$

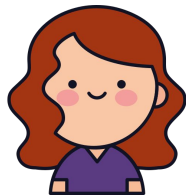
$$K = n_b \times P_a$$

Troca de chaves em ECC

- **Corretude:** $K = n_a \times P_b = n_a \times (n_b \times G) = n_b \times (n_a \times G) = n_b \times P_a$
 - portanto, ambos calculam a mesma chave K
- **Segurança:** um atacante deveria ser capaz de calcular n , dado G e nG .
 - com esse valor secreto n , ele seria capaz de calcular K
 - encontrar n significa resolver o problema do logaritmo discreto em curvas elípticas

$$P_a = n_a \times G$$

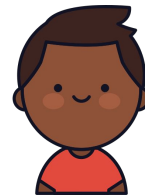
$$K = n_a \times P_b$$



n_a

P_a

P_b



n_b

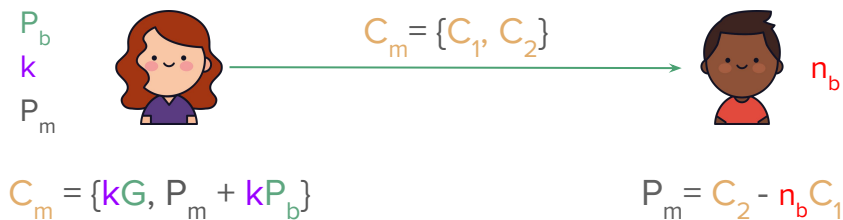
$$P_b = n_b \times G$$

$$K = n_b \times P_a$$

Cifragem e decifragem

Geração de chaves:

- Considere um ponto $G = (x_1, y_1)$ em $E_q(a,b)$
- Bob calcula:
 - chave privada n_b
 - chave publica $P_b = n_b \times G$



Cifragem:

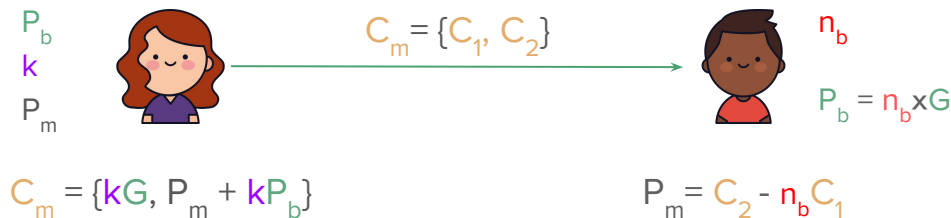
- Alice escolhe um valor secreto k e codifica a mensagem m em um ponto P_m
- Alice cifra a mensagem usando P_b
- $C_m = \{kG, P_m + kP_b\} = \{C_1, C_2\}$

Decifragem:

- Bob **decifra** a mensagem usando a sua chave secreta n_b
- $P_m = C_2 - n_b C_1$

Cifragem e decifragem

- **Corretude:** $C_2 - n_b C_1 = (P_m + kP_b) - n_b(kG) = P_m + k(n_b G) - n_b(kG) = P_m$
 - ao utilizar a chave secreta n_b , é possível recuperar a mensagem original P_m
- **Segurança:** Alice mascarou a mensagem P_m ao adicionar kP_b a ela.
 - para remover kP_b e obter P_m , um atacante precisaria descobrir k dado kP_b
 - descobrir k significa resolver o problema do logaritmo discreto em curvas elípticas



Segurança e Eficiência

- A segurança de ECC depende da dificuldade de determinar k dado kP e P
 - Elliptic Curve Discrete Logarithm Problem (ECDLP)
- O ataque mais rápido é conhecido como método *Pollard rho*
- Recomendação do **NIST** é a de utilizar chaves entre 256 e 512 bits para ECC
- ECC requer chaves muito menores para o mesmo nível de segurança do RSA
- Por usar chaves menores, requer menos poder computacional
 - ideal para dispositivos limitados

Curvas do NIST

- Quais curvas elípticas devemos utilizar?
- O SP 800-186 traz a medida de segurança e recomendação de curvas seguras
- As curvas são identificadas por um nome, como P-256, Edwards448, Curve448 e suas especificações são dadas no documento
- Através destes nomes, conseguimos identificá-las nas bibliotecas criptográficas

Sumário

- Contexto histórico e definições básicas
- Algoritmo de troca de chaves DH
- Criptossistema RSA
- Criptografia em curvas elípticas (ECC)
- **Considerações finais**
- Atividades práticas

Comparação do tamanho de chaves

Table 10.3 Comparable Key Sizes in Terms of Computational Effort for Cryptanalysis (NIST SP-800-57)

Symmetric Key Algorithms	Diffie–Hellman, Digital Signature Algorithm	RSA (size of n in bits)	ECC (modulus size in bits)
80	$L = 1024$ $N = 160$	1024	160–223
112	$L = 2048$ $N = 224$	2048	224–255
128	$L = 3072$ $N = 256$	3072	256–383
192	$L = 7680$ $N = 384$	7680	384–511
256	$L = 15,360$ $N = 512$	15,360	512+

Note: L = size of public key, N = size of private key.

Imagem: W. Stallings. *Cryptography and network security*. Cap 10.4

[NIST SP 800-57](#)

Recomendações de uso

- O [SP 800-57](#) recomenda que, a partir de 2031, o RSA seja usado com chaves a partir dos 3072 bits e ECC com chaves entre 256 e 512 bits.

Table 4: Security strength time frames

Security Strength		Through 2030	2031 and Beyond
< 112	Applying protection	Disallowed	
	Processing	Legacy use	
112	Applying protection	Acceptable	Disallowed
	Processing		Legacy use
128	Applying protection	Acceptable	Acceptable
192	and processing	Acceptable	Acceptable
256	information that is already protected	Acceptable	Acceptable

Considerações importantes

- Criptografia de chave pública não tornou a criptografia simétrica obsoleta
 - por causa do *overhead* computacional da criptografia de chave pública, a criptografia de chave simétrica muito dificilmente será abandonada
- Criptografia de chave pública não é mais segura que a simétrica
 - segurança depende do tamanho da chave e do esforço computacional necessário para quebrar uma cifra. Um esquema não é superior que o outro
- Distribuição de chaves não se tornou trivial com criptografia de chaves públicas
 - ainda são necessários protocolos que podem envolver uma autoridade central.

Sumário

- Contexto histórico e definições básicas
- Algoritmo de troca de chaves DH
- Criptossistema RSA
- Criptografia em curvas elípticas (ECC)
- Considerações finais
- **Atividades práticas**

Atividade: gerando chaves DH com openssl

- Gerar parâmetros públicos do DH:

```
openssl dhparam -out dhparams.pem 2048
```

- Visualizar os parâmetros:

```
openssl pkeyparam -in dhparams.pem -text -noout
```

- Gerar os valores públicos e privados da Alice:

```
openssl genpkey -paramfile dhparams.pem -out alice_keys.pem
```

- Visualizar os valores da Alice:

```
openssl pkey -in alice_keys.pem -text -noout
```

- Gerar os valores públicos e privados do Bob:

```
openssl genpkey -paramfile dhparams.pem -out bob_keys.pem
```

- Extrair e visualizar a chave pública da Alice para ser enviada ao Bob:

```
openssl pkey -in alice_keys.pem -pubout -out alice_pub.pem  
openssl pkey -pubin -in alice_pub.pem -text
```

Atividade: gerando chaves DH com openssl

- Gere a chave secreta (K) da Alice usando a sua chave privada (Xa) e a chave pública do Bob (Yb)

```
openssl pkeyutl -derive -inkey alice_keys.pem -peerkey bob_pub.pem -out alice_secret.bin
```

- Gere a chave secreta do Bob (K) usando a sua chave privada (Xb) e a chave pública da Alice (Yb)

```
openssl pkeyutl -derive -inkey bob_keys.pem -peerkey alice_pub.pem -out bob_secret.bin
```

- As duas chaves geradas devem ser iguais

```
cmp alice_secret.bin bob_secret.bin
```

Atividade: cifragem e decifragem RSA com openssl

- Gere a chave privada RSA e salve ela cifrada com o AES (não esqueça a senha!)

```
openssl genrsa -aes256 -out seunome.privada.pem 2048
```

- Visualize a chave gerada:

```
openssl pkey -in seunome.privada.pem -text
```

- Derive a chave pública a partir da chave privada

```
openssl rsa -pubout -in seunome.privada.pem -out seunome.publica.pem
```

- Visualize a chave gerada:

```
openssl pkey -pubin -in seunome.publica.pem -text
```

- **Pergunta:** qual a diferença de informações informadas ao visualizar a chave pública e a chave privada?

Atividade: cifragem e decifragem RSA com openssl

- Crie um arquivo de texto

```
echo "Esta é uma mensagem confidencial" > msg.txt
```

- Cifre o arquivo de texto com a chave pública

```
openssl pkeyutl -encrypt -inkey seunome.publica.pem -pubin -in msg.txt -out secretMsg.bin
```

- Decifre o arquivo com a chave privada

```
openssl pkeyutl -decrypt -inkey seunome.privada.pem -in secretMsg.bin -out msgDecifrada.txt
```

- **Pergunta:** Quem pode cifrar mensagens? E quem pode decifrar?

Atividade: Gerando chaves ECC com o openssl

- Verifique as curvas disponíveis no openssl:

```
openssl ecparam -list_curves
```

- Gere a chave privada com a curva escolhida:

```
openssl ecparam -genkey -name <nome_da_curva> -out <nome_chave_privada>.pem
```

- Extraia a chave pública:

```
openssl ec -in <nome_chave_privada>.pem -pubout -out <nome_chave_pública>.pem
```

- Visualize as chaves:

```
openssl ec -in <nome_chave_privada>.pem -text -noout
```

```
openssl ec -in <nome_chave_pública>.pem -text -noout -pubin
```

- Compare as chaves geradas com as chaves do RSA

- Obs: se escolher secp224r1 terá uma chave com o mesmo nível de segurança do RSA 2048. Compare os tamanhos das chaves.

Referências

- W. Stallings. *Cryptography and network security*. 7a edição.
 - Princípios de criptosistemas de chave pública: 9.1
 - RSA: 9.2
 - Diffie-Hellman: 10.1
- D. Stinson e M. Paterson. *Cryptography: Theory and Practice*. 4a edição.
 - Introdução a criptosistemas de chave pública: 6.1
 - RSA: 6.3
- Recomendações de tamanho de chaves para DH:
 - [NIST SP 800-56A](#)
 - [NIST SP 800-131A](#)
- DH usando openssl: <https://tinyurl.com/yzpjs9>
- [SP 800-57](#)
- [SP 800-186](#)
- PQC: [NIST Internal Report 8547](#)
- imagens: Flaticon.com